

])

```
X = A.inv() * B
```

```
print(X) # Matrix([[2], [34], [-8]])
```

Here is the solution in mathematical notation:

$$A^{-1}B = X$$

$$\begin{bmatrix} -\frac{1}{2} & 0 & \frac{1}{3} \\ 5.5 & -2 & \frac{4}{3} \\ -2 & 1 & \frac{1}{3} \end{bmatrix} \begin{bmatrix} 44 \\ 56 \\ 72 \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

$$\begin{bmatrix} 2 \\ 34 \\ -8 \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

Hopefully, this gave you an intuition for inverse matrices and how they can be used to solve a system of equations.



Systems of Equations in Linear Programming

This method of solving systems of equations is used for linear programming as well, where inequalities define constraints and an objective is minimized/maximized.

PatrickJMT has a lot of good videos on Linear Programming. We also cover it briefly in [Appendix A](#).

In practicality, you should rarely find it necessary to calculate inverse matrices by hand and can have a computer do it for you. But if you have a need or are curious, you will want to learn about Gaussian elimination. [PatrickJMT on YouTube](#) has a number of videos demonstrating Gaussian elimination.

Eigenvectors and Eigenvalues

Matrix decomposition is breaking up a matrix into its basic components, much like factoring numbers (e.g., 10 can be factored to 2×5).

Matrix decomposition is helpful for tasks like finding inverse matrices and calculating determinants, as well as linear regression. There are many ways to decompose a

matrix depending on your task. In [Chapter 5](#) we will use a matrix decomposition technique, QR decomposition, to perform a linear regression.

But in this chapter let's focus on a common method called eigendecomposition, which is often used for machine learning and principal component analysis. At this level we do not have the bandwidth to dive into each of these applications. For now, just know eigendecomposition is helpful for breaking up a matrix into components that are easier to work with in different machine learning tasks. Note also it only works on square matrices.

In eigendecomposition, there are two components: the eigenvalues denoted by λ and eigenvector by v shown in [Figure 4-24](#).

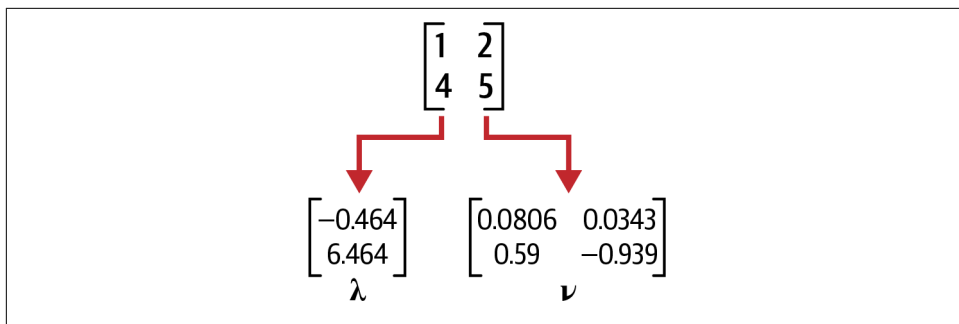


Figure 4-24. The eigenvector and eigenvalues

If we have a square matrix A , it has the following eigenvalue equation:

$$Av = \lambda v$$

If A is the original matrix, it is composed of eigenvector v and eigenvalue λ . There is one eigenvector and eigenvalue for each dimension of the parent matrix, and not all matrices can be decomposed into an eigenvector and eigenvalue. Sometimes complex (imaginary) numbers will even result.

[Example 4-20](#) is how we calculate eigenvectors and eigenvalues in NumPy for a given matrix A .

Example 4-20. Performing eigendecomposition in NumPy

```
from numpy import array, diag
from numpy.linalg import eig, inv

A = array([
    [1, 2],
    [4, 5]
])
```

```

eigenvals, eigenvecs = eig(A)

print("EIGENVALUES")
print(eigenvals)
print("\nEIGENVECTORS")
print(eigenvecs)

"""
EIGENVALUES
[-0.46410162  6.46410162]

EIGENVECTORS
[[-0.80689822 -0.34372377]
 [ 0.59069049 -0.9390708 ]]
"""

```

So how do we rebuild matrix A from the eigenvectors and eigenvalues? Recall this formula:

$$Av = \lambda v$$

We need to make a few tweaks to the formula to reconstruct A :

$$A = Q\Lambda Q^{-1}$$

In this new formula, Q is the eigenvectors, Λ is the eigenvalues in diagonal form, and Q^{-1} is the inverse matrix of Q . Diagonal form means the vector is padded into a matrix of zeroes and occupies the diagonal line in a similar pattern to an identity matrix.

Example 4-21 brings the example full circle in Python, starting with decomposing the matrix and then recomposing it.

Example 4-21. Decomposing and recomposing a matrix in NumPy

```

from numpy import array, diag
from numpy.linalg import eig, inv

A = array([
    [1, 2],
    [4, 5]
])

eigenvals, eigenvecs = eig(A)

print("EIGENVALUES")

```

```

print(eigenvals)
print("\nEIGENVECTORS")
print(eigenvecs)

print("\nREBUILD MATRIX")
Q = eigenvecs
R = inv(Q)

L = diag(eigenvals)
B = Q @ L @ R

print(B)

"""
EIGENVALUES
[-0.46410162  6.46410162]

EIGENVECTORS
[[-0.80689822 -0.34372377]
 [ 0.59069049 -0.9390708 ]]

REBUILD MATRIX
[[1.  2.]
 [4.  5.]]
"""

```

As you can see, the matrix we rebuilt is the one we started with.

Conclusion

Linear algebra can be maddeningly abstract and it is full of mysteries and ideas to ponder. You may find the whole topic is one big rabbit hole, and you would be right! However, it is a good idea to continue being curious about it if you want to have a long, successful data science career. It is the foundation for statistical computing, machine learning, and other applied data science areas. Ultimately, it is the foundation for computer science in general. You can certainly get away with not knowing it for a while but you will encounter limitations in your understanding at some point.

You may wonder how these ideas are practical as they may feel theoretical. Do not worry; we will see some practical applications throughout this book. But the theory and geometric interpretations are important to have intuition when you work with data, and by understanding linear transformations visually you are prepared to take on more advanced concepts that may be thrown at you later in your pursuits.